38

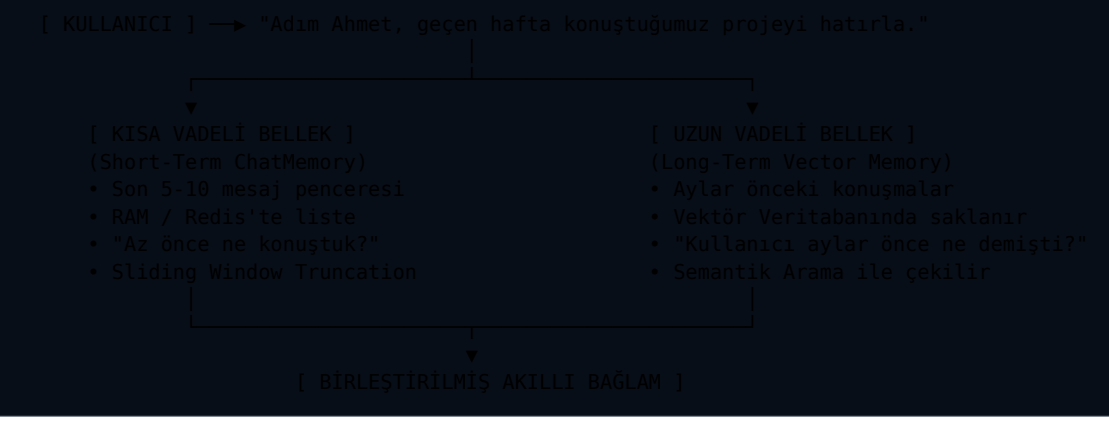
BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

Kısa ve Uzun Vadeli Bellek: ChatMemory vs Vector Memory

Memory Architectures: Short-Term (Sliding Window ChatMemory) vs Long-Term (Vector Memory)

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İnsan beynini düşünün: Çalışma belleğiniz (Kısa Vadeli Bellek), şu an bu cümleyi okurken aklınızda tuttuğunuz kelimelerdir. 10 dakika sonra bu kelimeleri unutursunuz ama ana fikri uzun vadeli hafızanıza atarsınız. 1 yıl sonra bir arkadaşınızı gördüğünüzde ise uzun vadeli hafızanızdan 'Bu Ahmet, geçen yaz tatile gitmiştik' diye anıyı geri çağırırsınız (Vektör Belleği). Yapay zekada da son 5 mesajı Redis'te tutmak kısa vadeli bellektir; aylar önceki tercihleri vektör veritabanında saklayıp gerektiğinde aramak uzun vadeli bellektir!

🇬🇧 ENGLISH (ELI5)

Consider human cognition: Short-term working memory holds the specific words you read 10 seconds ago. After the conversation ends, transient words fade, but core facts consolidate into your long-term hippocampal memory. When you meet an acquaintance a year later, your brain executes a semantic memory retrieval: 'Ah, this is Alex from the London conference.' In AI architectures, short-term memory is a fast Redis list holding the last N turns; long-term memory is a vector store indexing lifetime user facts!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Bellek Mimarisi Mühendisliği: 1) Short-Term (Sliding Window ChatMemory): `chatClient` her çağrıldığında son N mesajı (ör. son 10 mesaj) prompt'a ekler. Token sınırı doldukça en eski mesajlar listeden düşer (FIFO). 2) Summary Memory: Eski mesajlar silinmek yerine arka planda küçük bir modelle (GPT-4o-mini) 2 paragraflık bir 'Özet' haline getirilir. 3) Long-Term Vector Memory: Kullanıcının kalıcı tercihleri ('Fıstığa alerjim var', 'TypeScript severim') entity extraction ile yakalanır, vektörleştirilir ve kullanıcı ID'si ile indekslenir. İlgili konu açıldığında RAG ile çekilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Memory Engineering Paradigms: 1) Sliding Window (Short-Term): Buffers the last K conversational turns. As tokens breach a budget threshold, oldest turns are evicted (FIFO). 2) Conversational Summary Buffer: As history lengthens, background workers compress evicted turns into an evolving running summary prepended to the system prompt. 3) Long-Term Episodic Vector Memory: Fact extraction pipelines parse dialogues for user preferences ('I prefer dark mode and Java 25'), project them into a persistent vector store partitioned by `user_id`, and retrieve them on-demand via semantic similarity.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: Sliding Window ChatMemory ve Redis Entegrasyonu
import org.springframework.ai.chat.memory.ChatMemory;
import org.springframework.ai.chat.memory.InMemoryChatMemory;
import org.springframework.ai.chat.messages.Message;
import java.util.List;

public class ConversationMemoryManager {
    private final ChatMemory chatMemory;
    private final int maxMessagesWindow;

    public ConversationMemoryManager(int maxMessagesWindow) {
        this.chatMemory = new InMemoryChatMemory(); // Üretimde RedisChatMemory kullanılır
        this.maxMessagesWindow = maxMessagesWindow;
    }

    public List<Message> getRecentHistory(String conversationId) {
        List<Message> allMessages = chatMemory.get(conversationId, 100);
        if (allMessages.size() <= maxMessagesWindow) {
            return allMessages;
        }
        // Sliding window: Sadece son maxMessagesWindow kadar mesajı al
        return allMessages.subList(allMessages.size() - maxMessagesWindow, allMessages.size());
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Bütçe ve Güvenlik Tuzağı: Kullanıcının 50 mesajlık sohbet geçmişini filtrelemeden olduğu gibi her isteğe eklerseniz, hem her istekte 10.000 token harcarsınız hem de kullanıcının 20 mesaj önce yaptığı bir şaka modeli halüsinasyona sürükleyebilir. Sohbet geçmişi daima token sınırlı bir pencere (Window) ile tutulmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Context blowout trap: Appending infinite conversational history without bounds exponentially scales token costs and dilutes model attention. Always enforce strict sliding window limits or automated summarization buffers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Kısa vadeli bellek (Sliding Window), anlık diyalog akışını Redis/RAM üzerinde tutar.
- Uzun vadeli bellek (Vector Memory), kullanıcı tercihlerini semantik olarak arşivler.
- Eski diyalogları periyodik olarak özetlemek (Summarization), bağlamı ve maliyeti korur.

🎯 Key Takeaways for the Interview

- Short-term memory buffers active conversational turns in low-latency stores.
- Long-term episodic memory archives enduring user preferences in vector stores.
- Periodic conversation summarization preserves context without token explosion.